REALTHREAD / TECHNICAL DOCUMENTATION

GD32H77D-Gino

Technical Documentation

Guides and examples for the GD32H77D Gino board.

Document version **1.0.0**

Last updated 2026-10-21

2026, CYFS

Contents

Developer Guides

1. GD32H77D Gino SDK User Guide . . . 1
   - 1.1 Platform and Tools . . . 1
   - 1.2 Hardware Map . . . 2
   - 1.3 Memory and Download Layout . . . 3
   - 1.4 RT-Thread Studio . . . 3
   - 1.5 Env and MDK . . . 4
   - 1.6 Example Operation . . . 4

Getting Started

1. Gino_template . . . 7
   - 1.1 Introduction . . . 7
   - 1.2 GPIO and RTOS Scheduling Details . . . 7
   - 1.3 GD32H77D GPIO and SysTick Features . . . 7
   - 1.4 RT-Thread PIN and Thread Device Interface . . . 8
   - 1.5 Hardware . . . 8
   - 1.6 Example . . . 8
   - 1.7 Runtime Results . . . 9

Peripheral Drivers

1. Gino_driver_can . . . 10
   - 1.1 Introduction . . . 10
   - 1.2 CAN Arbitration and Error Handling Details . . . 10
   - 1.3 GD32H77D CAN1 Features . . . 11
   - 1.4 RT-Thread CAN Device Interface . . . 11
   - 1.5 Hardware . . . 11
   - 1.6 Example . . . 11
   - 1.7 Runtime Results . . . 14
2. Gino_driver_eth . . . 15
   - 2.1 Introduction . . . 15
   - 2.2 Ethernet, RMII, and TCP/IP Layers Details . . . 15
   - 2.3 GD32H77D ENET1 and RMII Features . . . 16
   - 2.4 RT-Thread Ethernet, Netdev, and SAL Device Interface . . . 16
   - 2.5 Hardware . . . 16
   - 2.6 Example . . . 16
   - 2.7 Runtime Results . . . 17
3. Gino_driver_i2c . . . 18
   - 3.1 Introduction . . . 18
   - 3.2 I2C Multi-Master/Slave Bus Protocol Details . . . 18
   - 3.3 GD32H77D I2C1 Features . . . 19
   - 3.4 RT-Thread I2C Bus Device Interface . . . 19
   - 3.5 Hardware . . . 19
   - 3.6 Example . . . 20
   - 3.7 Runtime Results . . . 21
4. Gino_driver_pwm . . . 22
   - 4.1 Introduction . . . 22
   - 4.2 PWM Timer Output Details . . . 22

4.3 GD32H77D Timer PWM Features . . . 22
4.4 RT-Thread PWM Device Interface . . . 23
4.5 Hardware . . . 23
4.6 Example . . . 23
4.7 Runtime Results . . . 24
5. Gino_driver_qspi_flash . . . 25
5.1 Introduction . . . 25
5.2 QSPI NOR Flash and Filesystem Details . . . 25
5.3 GD32H77D OSPI0 and MDMA Features . . . 25
5.4 RT-Thread FAL, Block Device, and DFS Device Interface . . . 26
5.5 Hardware . . . 26
5.6 Example . . . 26
5.7 Runtime Results . . . 27
6. Gino_driver_rtc . . . 28
6.1 Introduction . . . 28
6.2 RTC and Alarm Details . . . 28
6.3 GD32H77D RTC and Backup Domain Features . . . 29
6.4 RT-Thread RTC and Alarm Device Interface . . . 29
6.5 Hardware . . . 29
6.6 Example . . . 29
6.7 Runtime Results . . . 30
7. Gino_driver_sdcard . . . 31
7.1 Introduction . . . 31
7.2 SD/SDIO Block Protocol Details . . . 31
7.3 GD32H77D SDIO1 Features . . . 31
7.4 RT-Thread SDIO, Block Device, and DFS Device Interface . . . 32
7.5 Hardware . . . 32
7.6 Example . . . 32
7.7 Runtime Results . . . 33
8. Gino_driver_sdram . . . 34
8.1 Introduction . . . 34
8.2 SDRAM Timing, Refresh, and Heap Extension Details . . . 34
8.3 GD32H77D EXMC and SDRAM Features . . . 35
8.4 RT-Thread Memory Management Device Interface . . . 35
8.5 Hardware . . . 35
8.6 Example . . . 35
8.7 Runtime Results . . . 36
9. Gino_driver_spi . . . 37
9.1 Introduction . . . 37
9.2 SPI Synchronous Serial Protocol Details . . . 37
9.3 GD32H77D SPI3 Features . . . 38
9.4 RT-Thread SPI Bus Device Interface . . . 38
9.5 Hardware . . . 38
9.6 Example . . . 38
9.7 Runtime Results . . . 39
10. Gino_driver_usb_device . . . 40
10.1 Introduction . . . 40
10.2 USB Device Enumeration and CDC Details . . . 40
10.3 GD32H77D USBHS0 Device Features . . . 41
10.4 RT-Thread USB Device Interface . . . 41
10.5 Hardware . . . 41
10.6 Example . . . 41
10.7 Runtime Results . . . 42

11. Gino_driver_usb_host . . . 43
11.1 Introduction . . . 43
11.2 USB Host Enumeration and Mass Storage Details . . . 43
11.3 GD32H77D USBHS1 Host Features . . . 44
11.4 RT-Thread USB Host and DFS Device Interface . . . 44
11.5 Hardware . . . 44
11.6 Example . . . 44
11.7 Runtime Results . . . 45

Display and Camera

1. Gino_display_camera . . . 46
1.1 Introduction . . . 46
1.2 Capture and Display . . . 46
1.3 Memory Requirements . . . 46
1.4 Device Interfaces . . . 46
1.5 Hardware . . . 47
1.6 Example Operation . . . 47
1.7 Expected Behavior . . . 48
2. Gino_display_gd32embeddedui . . . 49
2.1 Editable UI package . . . 49
2.2 Hardware and initialization . . . 49
2.3 Open the board project . . . 49
2.4 Regenerate the UI . . . 50
2.5 Board checks . . . 50
3. Gino_display_lvgl . . . 51
3.1 Introduction . . . 51
3.2 LVGL Rendering, Input, and Double Buffering Details . . . 51
3.3 GD32H77D SDRAM, TLI/DSI, and I2C3 Features . . . 51
3.4 RT-Thread LVGL, Graphic, and Touch Device Interface . . . 52
3.5 Hardware . . . 52
3.6 Example . . . 53
3.7 Runtime Results . . . 54
4. Gino_display_lvgl8 . . . 55
4.1 Introduction . . . 55
4.2 LVGL Rendering, Input, and Double Buffering Details . . . 55
4.3 GD32H77D SDRAM, TLI/DSI, and I2C3 Features . . . 55
4.4 RT-Thread LVGL, Graphic, and Touch Device Interface . . . 56
4.5 Hardware . . . 56
4.6 Example . . . 57
4.7 Runtime Results . . . 58

Components

1. Gino_component_mqtt . . . 59
1.1 Introduction . . . 59
1.2 MQTT Publish/Subscribe Protocol Details . . . 61
1.3 GD32H77D UART4 and Cortex-M7 Features . . . 62
1.4 RT-Thread AT Device, SAL, and MQTT Device Interface . . . 62
1.5 Hardware . . . 62
1.6 Example . . . 62
1.7 Runtime Results . . . 63

Factory Demo

1. Gino_factory 64
1.1 Introduction 64
1.2 Concurrent Peripheral System Integration Details 68
1.3 GD32H77D Multiple Buses, DMA Engines, and Memories Features 68
1.4 RT-Thread Integrated Devices and Components Device Interface 68
1.5 Hardware 68
1.6 Example 69
1.7 Runtime Results 70

Developer Guides

1. GD32H77D Gino SDK User Guide

中文 | SDK overview

This guide covers the SDK directory layout, project selection, hardware connections, project creation, builds, downloads, and example operation.

1.1 Platform and Tools

- Board: GD32H77D Gino; MCU: GD32H77DIW, Arm Cortex-M7, up to 600 MHz.
- RT-Thread: 5.3.0.
- RT-Thread Studio toolchain: GNU Arm Embedded 13.3.
- Env: version 2.0 or later, with a configured GNU Arm Embedded toolchain.
- MDK: Keil MDK 5.43 or compatible, Arm Compiler 6, GigaDevice GD32H77x_78x_DFP 0.6.1, and either DAP-Link (CMSIS-DAP) or J-Link.
- Download connection: SWD; Studio supports DAP-Link/PyOCD and J-Link; MDK supports DAP-Link (CMSIS-DAP) and J-Link.
- J-Link: the software version currently used is v9.76a.
- Console: UART1, PA2/PA3, AF7, 115200-8-N-1.
- Projects: 17, including Gino_template and 16 examples.

SDK Directory Layout

The examples share the RT-Thread kernel, GD32 libraries, board drivers, and offline packages. Paths below are relative to the SDK root.

Directory	Contents
projects/<name>/applications	Application entry point and example code
projects/<name>/board	Project board configuration and linker scripts
libraries/Board_Drivers	Shared board initialization and onboard peripheral drivers
libraries/gd32_drivers	GD32 peripheral drivers for RT-Thread
libraries/gd32-arm-*	GD32 CMSIS, startup code, and peripheral libraries
rt-thread	RT-Thread kernel, components, and build tools
packages	Offline packages used by the examples

Each project has its own .config, rtconfig.h, Studio metadata, and MDK project files. Configure features in the selected project; changes to shared drivers affect all projects that use them.

For a first build or a new application, start with Gino_template. To test a peripheral, select its independent example from section 6. Use Gino_factory for the integrated board demonstration.

1.2 Hardware Map

Peripheral	Pins or resource	Notes
Run LED	PC4	toggled every 500 ms
UART1	PA2/PA3, AF7	console
UART4	PB12/PB13, AF14	GD32VW553 AT module
I2C1	PH4/PB11, AF4	independent I2C example
I2C2	PA8/PC9, AF4	OV7670 control
I2C3	PD12/PD13, AF4	GT911 touch
SPI3	PE12/PF0/PF1	SCK/MISO/MOSI
CAN1	PB4/PB5	external CAN transceiver and termination required
LCD backlight	TIMER1 CH0, PA0	pwm1 channel 1
PWM outputs	PA7 and PG7	pwm2 channel 2 and pwm30 channel 3
GT911	reset PH3, interrupt PH5	address 0x5D
OV7670	XCLK PE9, I2C2, DCI + DMA1 channel 7	QVGA RGB565
QSPI flash	CS PC11, CLK PF10, D0-D3 PF8/PF9/PF7/PA1	8 MiB GD25Q64E on OSPI0
USB device	USBHS0, PA11/PA12	Type-C
USB host	USBHS1, PB14/PB15, VBUS PF2	USB-A
Ethernet	ENET1 RMII	PHY address 2, 50 MHz REF_CLK on PC12
SDRAM	0xC0000000, 32 MiB	display buffers and extended heap

Follow the corresponding project README for external module power, common ground, and interface voltage requirements.

1.3 Memory and Download Layout

Region	Address	Size	Use
CNVM	0x08010000	1984 KiB	application vector table, startup, and primary code
ECNVM	0x08200000	7680 KiB	extended program flash, as allocated by the linker script
ITCMRAM	0x00000000	128 KiB	allocated by the linker script
DTCMRAM	0x20000000	256 KiB	allocated by the linker script
AXISRAM	0x24000000	768 KiB	data, BSS, stack, internal heap, and camera buffers
SRAM1	0x30004000	16 KiB	Ethernet DMA descriptors and buffers
SDRAM	0xC0000000	32 MiB	display buffers and extended heap

The application vector table uses 0x08010000; a boot entry at 0x08000000 provides compatibility with different chip revisions. The default BIN image includes this boot entry and places the application vector table at offset 0x10000. Download the complete BIN image at 0x08000000.

With BSP_USING_LCD_MIPI enabled, the first 3 MiB of SDRAM are reserved and the extended heap starts at 0xC0300000. Without LCD, the SDRAM heap starts at 0xC0000000. Camera capture and preview buffers use internal AXI SRAM, separately from the SDRAM display area.

1.4 RT-Thread Studio

1. Install the GD32H77D-Gino BSP **v1.0.0** described by sdk-bsp-gd32h77d-gino.yaml.
2. Create an RT-Thread project based on the GD32H77D-Gino board.
3. Select Gino_template or one of the 16 examples.
4. Configure peripherals and packages in RT-Thread Settings, then build with GNU Arm Embedded 13.3.
5. Connect UART1 at 115200-8-N-1 and the SWD download adapter.
6. Select DAP-Link, target GD32H77DIW, and BIN mode; set the download start address to 0x08000000 and download Debug/rtthread.bin. These are the SDK’s default download settings. Builds retain Debug/rtthread.elf for debugging.

7. Reset the board and run the selected example as described below.

Studio-created projects contain the shared source trees required by the selected example.

1.5 Env and MDK

Env with GCC

In an Env PowerShell session, enter the chosen project from the SDK root. For example:

```
cd projects\Gino_template
.\mklinks.bat
$env:RTT_EXEC_PATH = "<gnu-arm-toolchain>\bin"
scons -j8
```

Replace the toolchain path with the directory containing arm-none-eabi-gcc. mklinks.bat creates links to the shared rt-thread and libraries directories. SCons produces rt-thread.elf, rtthread.hex, and rtthread.bin; download rtthread.bin at 0x08000000.

MDK5 with Arm Compiler 6

Install the MDK dependencies listed in section 1. In an Env PowerShell session, enter the chosen project from the SDK root:

```
cd projects\Gino_template
.\mklinks.bat
scons --target=mdk5
```

template.uvprojx and template.uvoptx hold the uVision defaults. SCons generates source groups, include paths, and defines from the current configuration. Regenerate the uVision project after changing configuration or source selection; generated source groups are overwritten.

1.6 Example Operation

The sidebar contains each example’ s README with its wiring, configuration, commands, and expected results. This table summarizes the prerequisites and first operation.

Project	Prerequisite	Operation or expected behavior
Gino_template	board power	PC4 blinks and the UART1 console is available
Gino_driver_i2c	pull-ups and an I2C target	i2c scan hwi2c1 08 78
Gino_driver_spi	SPI target and application-configured CS	use spi3; configure the target protocol in the application
Gino_driver_qspi_flash	onboard GD25Q64E	access files with ls /flash
Gino_driver_sdcard	FAT/FAT32 SD card	ls /sd
Gino_driver_can	CAN transceiver, termination, and peer	use can1; match bitrate and frame format to the peer
Gino_driver_pwm	external output connection as needed	pwm_dual_test start; PWM on PA7/PG7
Gino_driver_rtc	RTC clock source	date
Gino_driver_sdram	onboard SDRAM	external memory is added to the heap; free, list_memheap
Gino_driver_usb_device	data-capable USB cable	host enumerates the configured USB class
Gino_driver_usb_host	FAT USB drive	ls /udisk
Gino_driver_eth	Ethernet cable and network	ifconfig, ping <gateway>
Gino_display_lvgl	LCD, touch, and SDRAM	LVGL 9.x demo starts automatically; operate it through touch
Gino_display_lvgl8	LCD, touch, and SDRAM	LVGL 8.3.11 demo starts automatically; operate it through touch
Gino_display_camera	OV7670, LCD, and SDRAM	automatic preview; ov7670_preview start / ov7670_preview stop
Gino_component_mqtt	configured Wi-Fi and broker	gd32vw553_mqtt_start, gd32vw553_mqtt_pub <topic> <message>
Gino_factory	enabled board peripherals	operate the integrated display, camera, storage, and network examples

Flash Filesystem

The filesystem FAL partition is mounted at /flash. For a new partition that needs formatting:

```
mkfs -t elm filesystem
mount filesystem /flash elm
ls /flash
```

Formatting erases existing partition data. FatFs must allow a 4096-byte maximum sector size.

Wi-Fi and MQTT

Configure the Wi-Fi SSID/password, broker address/port, topics, client ID, and optional authentication in RT-Thread Settings or menuconfig. Keep real credentials in local configuration. Use a numeric IPv4 broker address when module firmware does not support DNS through AT+CIPDOMAIN.

Display and Camera

Gino_display_lvgl (9.x) and Gino_display_lvgl8 (8.3.11) start the LVGL music demo automatically. Gino_display_camera requests preview after LVGL initialization and displays QVGA RGB565 frames. Use ov7670_preview stop and ov7670_preview start to control preview. LCD framebuffers remain in the reserved SDRAM area; application allocations must use the configured heap.

Getting Started

1. Gino_template

1.1 Introduction

This project is the GD32H77D Gino development board reference project for LED blink and secondary development. It is used to learn, configure, and validate GPIO and RTOS scheduling separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

This project has no standalone primary device node.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check the runtime probe state used by this example.	This template has no standalone primary device node.
pin list	List PIN device information.	Confirm GPIO/PIN registration.

1.2 GPIO and RTOS Scheduling Details

After startup assembly initializes the data sections, RT-Thread configures clocks, console, system tick, and the scheduler, then enters the application's main. The application drives PC4 through the PIN device API and yields with rt_thread_mdelay; blinking therefore validates GPIO, tick timing, and scheduling together.

The complete data path is:

```
reset -> board init -> RT-Thread scheduler -> main thread -> rt_pin_write -> PC4
  LED
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

1.3 GD32H77D GPIO and SysTick Features

GD32H77D GPIO supports input, push-pull/open-drain output, and alternate functions. The run LED is on PC4. The BSP configures clocks, SysTick, the PIN driver, and UART1 in board.c, so the blink also checks tick and main-thread scheduling.

1.4 RT-Thread PIN and Thread Device Interface

The RT-Thread PIN API uses rt_pin_mode for direction and rt_pin_write/rt_pin_read for levels. The demo calls rt_thread_mdelay(500) to yield instead of busy-waiting.

This project has no standalone primary device node. Use list_device and gino_device_probe first to check the runtime state. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

1.5 Hardware

No additional hardware is required. The PC4 LED validates the system tick, GPIO, and basic startup.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

1.6 Example

Source paths below are relative to the project directory in the SDK repository:

- applications/main.c
- applications/device_probe.c
- ../../libraries/Board_Drivers/board.c
- board/SConscript

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_template
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- pin list

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

1.7 Runtime Results

Expected Behavior

- UART1 prints the RT-Thread banner and project name.
- PC4 toggles every 500 ms, giving an approximately 1 s on/off cycle.
- pin list shows the PIN device and pin information.

Peripheral Drivers

1. Gino_driver_can

1.1 Introduction

This project demonstrates external CAN1 communication on the GD32H77D Gino board. Startup initializes the device and reception without transmitting. Each gino_can_send command sends one classic CAN frame, followed by one CAN FD frame when enabled. The command can be called repeatedly. Received frames are printed to the UART console using colored ulog output.

Primary device: can1.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_can_send	Send one classic CAN frame, then one CAN FD frame when enabled.	Sends one set per invocation; repeat the command to send again.
gino_device_probe	Check whether the primary device used by this example is registered.	Checks can1 by default.
list_device	List registered system devices.	Confirm that UART, PIN, buses, and target devices exist.

1.2 CAN Arbitration and Error Handling Details

CAN1 handles bit timing, ID arbitration, ACK, error counters, and retransmission. PB4/PB5 still require an external transceiver for CANH/CANL. Lower numeric IDs normally win arbitration, while the application defines the payload protocol.

The complete data path is:

```
application CAN frame -> RT-Thread can1 -> CAN1 -> PB4/PB5 -> transceiver ->
  CANH/CANL
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

1.3 GD32H77D CAN1 Features

The project enables GD32H77D CAN1 on PB4/PB5. The controller provides bit timing, filters, TX mailboxes, RX FIFOs, error counters, and retransmission; an external transceiver supplies the differential physical layer.

1.4 RT-Thread CAN Device Interface

RT-Thread registers CAN1 as can1. Applications use rt_device_open to configure interrupt-driven TX/RX, rt_device_control to set bitrate and mode, and rt_device_read/rt_device_write to exchange struct rt_can_msg objects.

Primary device: can1. The application opens it in normal mode. The MSH command sends frames in order; the RX callback wakes a receive thread that drains the queue and prints each frame.

1.5 Hardware

CAN1 uses PB4/AF4 for TX and PB5/AF9 for RX. Connect the external transceiver to the peer's CANH/CANL and reference ground, with 120-ohm termination at each end of the bus. With CAN FD enabled, both the transceiver and peer must support ISO CAN FD at a 2 Mbit/s data rate.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

A CAN transceiver and proper termination are mandatory; MCU pins cannot connect directly to CANH/CANL.

1.6 Example

Reception starts after reset without transmitting. Run the following MSH command to send one set of frames:

```
msh /> gino_can_send
```

The command returns after sending once and can be invoked again. Reception remains active.

- applications/main.c: CAN initialization, ordered classic CAN/CAN FD transmission, and RX printing.
- applications/device_probe.c
- ../../libraries/gd32_drivers/drv_can.c
- ../../libraries/gd32_drivers/config/can_config.h

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_can
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Send Command

The arbitration bitrate is **500 kbit/s**. With RT_CAN_USING_CANFD defined, the application enables ISO CAN FD at **2 Mbit/s** for the data phase while also supporting classic frames.

Order	Format	Standard ID	Length	BRS	Payload
1	Classic CAN data frame	0x123	8 bytes	0	00 01 02 03 04 05 06 07
2	CAN FD data frame, only when enabled	0x124	64 bytes	1	Incrementing bytes from 00 through 3F

Each gino_can_send invocation sends one classic frame, waits 100 ms when CAN FD is enabled, then sends one FD frame and returns. Without CAN FD, only the classic frame is sent. Successful writes produce green [TX] logs through LOG_I; failed or timed-out writes produce red [TX FAIL] logs through LOG_E. The application does not automatically retry the command.

Receive Output

The receive thread prints each external frame as one green [RX] line through LOG_I, including CAN/CANFD format, standard/extended ID, RTR, BRS, byte count, and the complete hexadecimal payload. The interrupt callback only signals the thread.

The project enables RT_USING_ULOG, ULOG_BACKEND_USING_CONSOLE, and ULOG_USING_COLOR. Logs include tick time, level, and the can.demo tag. ULOG_LINE_BUF_SIZE=512 accommodates a complete 64-byte FD payload. Use a serial terminal with ANSI color support.

The board runs in normal mode and does not receive its own transmissions. Configure the peer to echo the frames, or manually send the same IDs and payloads, to see corresponding TX and RX lines. ACK alone does not produce an application RX frame. Incoming frames are printed without being automatically retransmitted.

Example log bodies when running the command with peer echo enabled, with ulog time, level, and tag prefixes omitted. TX/RX log ordering depends on the peer’ s response timing:

```
can1 normal mode: arbitration=500000 bit/s, ISO CAN FD data=2000000 bit/s
[TX] can1 CAN STD ID=0x00000123 RTR=0 BRS=0 LEN=8 DATA=00 01 02 03 04 05 06 07
[RX] can1 CAN STD ID=0x00000123 RTR=0 BRS=0 LEN=8 DATA=00 01 02 03 04 05 06 07
[TX] can1 CANFD STD ID=0x00000124 RTR=0 BRS=1 LEN=64 DATA=00 01 02 03 04 05 06 07
  08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F 20 21
  22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B
  3C 3D 3E 3F
[RX] can1 CANFD STD ID=0x00000124 RTR=0 BRS=1 LEN=64 DATA=00 01 02 03 04 05 06 07
  08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F 20 21
  22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B
  3C 3D 3E 3F
```

CAN FD Configuration

This project enables RT_CAN_USING_CANFD and has a 1152-byte non-blocking TX buffer for 16 rt_can_msg objects; the example uses blocking writes to preserve transmission order. Other projects can enable Enable CAN-FD support under RT-Thread Components -> Device Drivers -> Using CAN device drivers.

can_demo_init() configures enable_canfd according to the build option before opening the device for interrupt-driven TX/RX in normal mode. Classic frames use fd_frame=0; subsequent FD frames use fd_frame=1 and brs=1. The controller does not need to be reinitialized between the two frame types.

- message.len is a byte count: use 64 for 64 bytes, not the DLC encoding 15. Legal lengths are 0 through 8, 12, 16, 20, 24, 32, 48, and 64. Other lengths from 9 through 63 are zero-padded to the next legal length, which is reported on reception.
- fd_frame=0 sends a classic CAN frame of up to 8 bytes. fd_frame=1, brs=0 sends an FD frame without bitrate switching; fd_frame=1, brs=1 enables data-phase bitrate switching. CAN FD does not support remote frames.
- Reception still uses rt_device_read; fd_frame, brs, and len describe the actual frame. Set hdr_index=-1 before reading the general RX queue when hardware filtering is enabled.
- The maximum data rate is 8 Mbit/s. The driver calculates timing from the actual APB2 clock with a shared arbitration/data prescaler and a sample point near 80%. At the default 300 MHz APB2 clock, 500 kbit/s arbitration supports 2, 4, and 5 Mbit/s data rates. An exact 8 Mbit/s is unavailable at this clock and returns -RT_EINVAL. Custom bit timing currently returns -RT_ENOSYS.
- A 64-byte CAN FD mailbox occupies 72 bytes, so the 512-byte message RAM holds at most seven mailboxes. With CAN FD compiled in, mailboxes 0-2 receive and 3-6 transmit. RT_CANSND_BOX_NUM accepts 1-3, leaving at least one TX mailbox for non-blocking sends. Without CAN FD compiled in, the original 16 RX / 16 TX layout is retained.
- RT_CAN_CMD_SET_CANFD with (void *)0U restores classic CAN mode. Stop transmitting and drain the TX queues before changing configuration; active hardware TX returns -RT_EBUSY.

Validation

1. Connect the transceiver, CANH/CANL, reference ground, and termination. Open UART1 at 115200-8-N-1.
2. Set the external peer to normal mode at 500 kbit/s arbitration. With CAN FD enabled, also configure ISO CAN FD with a 2 Mbit/s data rate.
3. Reset the board; no frames are sent yet. Run gino_can_send and confirm that the peer receives classic ID 0x123, followed by FD ID 0x124 when enabled. Run the command again to send another set.
4. Send frames from the peer or enable peer echo, and check the complete [RX] data in the board's console. Other incoming IDs are also accepted.
5. Use list_device or gino_device_probe to inspect registration. For [TX FAIL], check the peer's ACK, bitrates, wiring, and termination.

1.7 Runtime Results

Expected Behavior

- list_device and gino_device_probe find can1.
- Startup sends no frames. Each gino_can_send invocation sends classic CAN ID 0x123, then CAN FD ID 0x124 when enabled.
- Frames sent or echoed by the peer appear as [RX] lines with their complete payloads.
- CANH/CANL show valid recessive levels while idle.

2. Gino_driver_eth

2.1 Introduction

This project is the GD32H77D Gino development board reference project for ENET1 RMII Ethernet. It is used to learn, configure, and validate Ethernet, RMII, and TCP/IP Layers separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: e0.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks e0 by default.
ifconfig	Check the connection state and IP address of the netdev.	Confirm that the network is ready.
ping 192.168.1.1	Send ICMP packets through the active network interface.	Replace the address with the actual gateway or peer.

2.2 Ethernet, RMII, and TCP/IP Layers Details

lwIP pbufs pass through RT-Thread eth_device to ENET1. TX copies pbuf chains and hands DMA descriptor ownership to hardware; RX interrupt only wakes the network thread, which copies valid frames and returns descriptors to DMA. A PHY thread handles address detection, autonegotiation, and link status.

The complete data path is:

```
socket/SAL -> lwIP -> e0 eth_device -> ENET DMA -> RMII -> PHY -> cable
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

2.3 GD32H77D ENET1 and RMII Features

The project uses GD32H77D ENET1 RMII with PHY address 2 and a 50 MHz REF_CLK on PC12. ENET DMA transfers frames through dedicated descriptors/buffers, while MDIO/MDC manages the PHY.

2.4 RT-Thread Ethernet, Netdev, and SAL Device Interface

The driver registers an RT-Thread eth_device and the e0 netdev. RX interrupts call eth_device_ready to wake the network thread, lwIP supplies IP/TCP/UDP, and SAL exposes sockets; ifconfig shows network configuration and ping checks reachability.

Primary device: e0. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

2.5 Hardware

ENET1 uses RMII with default PHY address 2. The PHY must provide a 50 MHz reference clock on PC12.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

2.6 Example

Source paths below are relative to the project directory in the SDK repository:

- ../../libraries/Board_Drivers/drv_enet.c
- board/SConscript
- rtconfig.h
- applications/device_probe.c

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_eth
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- ifconfig
- ping 192.168.1.1

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

2.7 Runtime Results

Expected Behavior

- The PHY is found at default address 2 and autonegotiates.
- ifconfig reports e0 link up and a valid IP.
- Gateway ping is stable without descriptor errors under traffic.

3. Gino_driver_i2c

3.1 Introduction

This project is the GD32H77D Gino development board reference project for the I2C1 hardware bus. It is used to learn, configure, and validate the I2C multi-master/slave bus protocol separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: hwi2c1.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks hwi2c1 by default.
i2c scan hwi2c1	Scan the I2C1 bus and print responding slave addresses.	Provided by i2c-tools; connect a powered I2C target first.

The project enables i2c-tools v1.0.0 (PKG_USING_I2C_TOOLS) in hardware bus mode. Run mklinks.bat on Windows or sh mklinks.sh on Linux to link the bundled package before opening the project. Run i2c for scan/read/write usage.

3.2 I2C Multi-Master/Slave Bus Protocol Details

I2C uses open-drain SDA/SCL lines with pull-ups. The controller generates START, address, ACK, data, and STOP. i2c scan uses address-only write transfers to check for ACK; it does not validate device registers.

The complete data path is:

```
MSH i2c scan -> i2c-tools -> hwi2c1 -> I2C1 controller -> PH4/PB11 -> target ACK
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

3.3 GD32H77D I2C1 Features

The project uses GD32H77D hardware I2C1 on PH4/PB11 AF4. The controller generates START/STOP, addressing, ACK/NACK, and byte timing; SDA/SCL still require pull-ups.

3.4 RT-Thread I2C Bus Device Interface

RT-Thread registers the controller as the hwi2c1 bus. Device drivers use rt_i2c_transfer or rt_i2c_master_send/rt_i2c_master_recv; i2c scan hwi2c1 scans address ACKs.

Primary device: hwi2c1. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

3.5 Hardware

I2C1 uses PH4/PB11 (AF4). The external bus requires suitable pull-up resistors.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

Use suitable pull-up resistors and keep a common reference with the external device.



3.6 Example

Source paths below are relative to the project directory in the SDK repository:

- applications/main.c
- applications/device_probe.c
- ../../libraries/gd32_drivers/drv_hard_i2c.c
- ../../libraries/gd32_drivers/config/i2c_config.h
- ../../packages/i2c-tools-v1.0.0/src/i2c_tools.c
- ../../packages/i2c-tools-v1.0.0/src/i2c_utils.c

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_i2c
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories. This project also links the bundled packages: i2c-tools-v1.0.0.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- i2c scan hwi2c1 08 78

The scan range is hexadecimal and excludes the stop address: 08 78 probes 0x08-0x77, skipping reserved addresses. Without a range, the package scans 0x00-0x7F. gino_device_probe only checks bus registration; it does not probe slave devices.

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

3.7 Runtime Results

Expected Behavior

- list_device shows hwi2c1.
- A connected target appears in i2c scan hwi2c1 08 78.
- A logic analyzer shows START, a 7-bit address, ACK, and STOP.

4. Gino_driver_pwm

4.1 Introduction

This project is the GD32H77D Gino development board reference project for PWM2 and PWM30 dual-channel output. It is used to learn, configure, and validate PWM Timer Output separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: pwm2.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks pwm2 by default.
pwm_dual_test start	Start the dual-channel PWM output test.	Outputs PWM on PA7 and PG7.
pwm_dual_test stop	Stop the dual-channel PWM output test.	Disables the test outputs.

4.2 PWM Timer Output Details

A timer period sets PWM frequency and a compare value sets high time. The demo drives pwm2 channel 2 and pwm30 channel 3 while changing pulse width, validating both timer channels and pin muxes.

The complete data path is:

```
pwm_dual_test -> RT-Thread PWM -> timer compare outputs -> PA7 and PG7
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

4.3 GD32H77D Timer PWM Features

GD32H77D timers use a counter period for PWM frequency and a compare value for active time. The project exposes pwm2 channel 2 on PA7 and pwm30 channel 3 on PG7.

4.4 RT-Thread PWM Device Interface

The RT-Thread PWM interface uses rt_device_pwm_set/rt_pwm_set to set period and pulse, rt_device_pwm_enable/rt_pwm_enable to start a channel, and the corresponding disable interface to stop it. Time units follow the active RT-Thread PWM API definition.

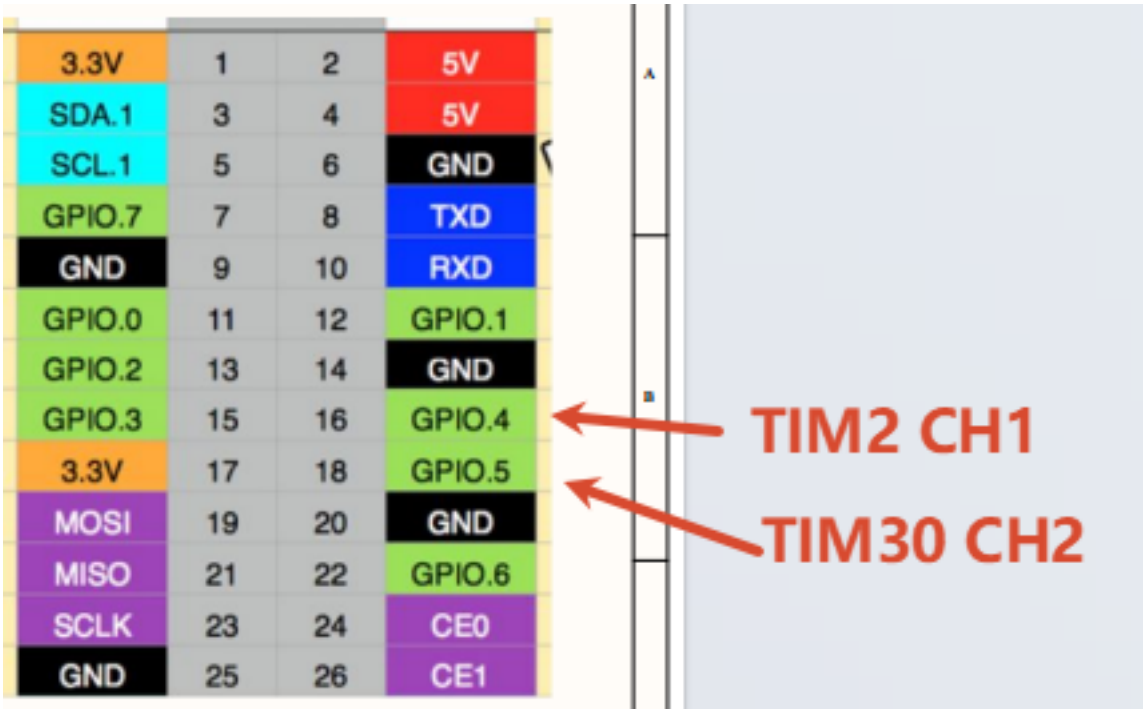
Primary device: pwm2. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

4.5 Hardware

pwm2 channel 2 is on PA7 and pwm30 channel 3 is on PG7; inspect them with an oscilloscope or logic analyzer.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

Use an oscilloscope or logic analyzer to confirm frequency and duty cycle on PA7 and PG7.



4.6 Example

Source paths below are relative to the project directory in the SDK repository:

- applications/pwm_dual_test.c
- ../../libraries/gd32_drivers/drv_pwm.c
- board/Kconfig

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_pwm
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- pwm_dual_test start
- pwm_dual_test stop

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

4.7 Runtime Results

Expected Behavior

- pwm2 and pwm30 appear in list_device.
- PA7 and PG7 output about 2 kHz after start.
- Duty cycle changes on an oscilloscope and outputs stop on command.

5. Gino_driver_qspi_flash

5.1 Introduction

This is the GD32H77D Gino QSPI flash reference project for validating the onboard GD25Q64E connected through OSPI0. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: qspi_flash0.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks qspi_flash0 by default.
fal probe filesystem	Probe the FAL filesystem partition.	Confirm partition geometry before filesystem access.
fal read 0 16	Read data from the current FAL device.	Use for a basic flash read check.

5.2 QSPI NOR Flash and Filesystem Details

OSPI0 uses quad data phases to access the GD25Q64E NOR flash. The driver supplies read/program/erase operations, FAL exposes the filesystem partition, and a block adapter lets DFS/FatFs mount it at /flash. NOR erase and busy-state rules still apply.

The complete data path is:

```
GD25Q64E -> OSPI0/MDMA -> qspi_flash0 -> FAL filesystem -> block device -> FatFs
  -> /flash
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

5.3 GD32H77D OSPI0 and MDMA Features

GD32H77D OSPI0 supports multi-line command/address/data phases and MDMA-assisted transfers. The onboard GD25Q64E is 8 MiB, with typical 4 KiB erase sectors and 256-byte program pages.

5.4 RT-Thread FAL, Block Device, and DFS Device Interface

The driver registers qspi_flash0. FAL uses fal_init and fal_partition_find to locate the filesystem partition, then exposes a block device through fal_blk_device_create, which DFS/FatFs mounts at /flash.

Primary device: qspi_flash0. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

5.5 Hardware

The onboard 8 MiB GD25Q64E is registered as qspi_flash0; the filesystem FAL partition is mounted at /flash and may need initial formatting.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

An unformatted partition may need formatting before /flash can mount; formatting destroys existing contents.

5.6 Example

Source paths below are relative to the project directory in the SDK repository:

- ../../libraries/Board_Drivers/drv_qspi_flash.c
- ../../libraries/Board_Drivers/drv_filesystem.c
- ../../libraries/Board_Drivers/fal_cfg.h
- ../../libraries/gd32_drivers/drv_ospi.c

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_qspi_flash
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- fal probe filesystem
- fal read 0 16

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

5.7 Runtime Results

Expected Behavior

- gino_device_probe finds qspi_flash0.
- fal probe filesystem reports partition geometry.
- fal read 0 16 is stable, and a formatted partition mounts at /flash.

6. Gino_driver_rtc

6.1 Introduction

This project is the GD32H77D Gino development board reference project for On-chip RTC and alarm. It is used to learn, configure, and validate RTC and Alarm separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: rtc.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks rtc by default.
date	Read or set the RTC time through the MSH date command.	Run repeatedly to confirm seconds advance.
rtc_alarm [seconds]	Start a one-shot RTC alarm.	Default: 5 seconds; range: 1..86400. Replaces the previous example alarm.
rtc_alarm status	Show the example alarm state and deadline.	Times are displayed in the local timezone.
rtc_alarm stop	Cancel and delete the example alarm.	Does not delete alarms owned by other applications.

6.2 RTC and Alarm Details

The RTC uses a low-speed clock and backup domain. RT-Thread exposes it as rtc and date accesses it through the common RTC API. Alarm events come from RTC compare logic instead of main-loop polling.

The complete data path is:

```
low-speed clock/backup domain -> on-chip RTC -> rt_device rtc -> date/alarm
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

6.3 GD32H77D RTC and Backup Domain Features

The on-chip RTC runs from the low-speed clock and backup domain and can continue while the CPU is stopped. Alarm compare logic can interrupt or wake the system; retention depends on backup-domain and VBAT conditions.

6.4 RT-Thread RTC and Alarm Device Interface

RT-Thread registers rtc. RTC device-control operations back POSIX time functions and the MSH date command. The alarm component manages alarm objects and callbacks.

Primary device: rtc. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

6.5 Hardware

Validates the on-chip RTC and alarm. Retention depends on the RTC clock, backup domain, and backup power.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

RTC retention depends on the low-speed clock, backup domain, and VBAT conditions.

6.6 Example

Source paths below are relative to the project directory in the SDK repository:

- applications/device_probe.c
- applications/rtc_alarm_example.c
- ../../libraries/gd32_drivers/drv_rtc.c
- board/Kconfig

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_rtc
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- date
- rtc_alarm 5
- rtc_alarm status
- rtc_alarm stop

```
date 2026 09 08 12 00 00
rtc_alarm 5
rtc_alarm status
```

After approximately five seconds, ulog prints a single [I/rtc.alarm] Triggered: message with the local date and time. rtc_alarm without arguments also schedules an alarm five seconds ahead; rtc_alarm help prints command usage. To test cancellation, run rtc_alarm 30 followed by rtc_alarm stop before it expires.

The command uses rt_alarm_create() and rt_alarm_start() in one-shot mode. The RTC Alarm0 interrupt notifies the RT-Thread Alarm service, which prints the callback log from its thread. The example enables the ulog console backend at INFO level and uses a 2048-byte Alarm service stack. Application messages also use ulog; the built-in date and list_device commands keep their standard shell output.

Alarm deadlines are derived from the RTC time; avoid changing date while an alarm is armed. The example alarm object is held in RAM and must be recreated after a reset, even when RTC time is retained.

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

6.7 Runtime Results

Expected Behavior

- gino_device_probe opens rtc.
- Repeated date calls show increasing seconds.
- rtc_alarm 5 produces one ulog trigger message; a stopped alarm does not trigger.
- Retention across reset matches the backup-domain and VBAT design.

7. Gino_driver_sdcard

7.1 Introduction

This project is the GD32H77D Gino development board reference project for SDIO1 and SD card filesystem. It is used to learn, configure, and validate SD/SDIO Block Protocol separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: sd0.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks sd0 by default.
ls /sd	List files on the mounted SD card filesystem.	Requires a usable FAT/FAT32 card.

7.2 SD/SDIO Block Protocol Details

After SDIO1 identifies the card and provides block I/O, it registers sd0. DFS/FatFs reads partition and filesystem metadata and mounts it at /sd. Card detect alone does not prove initialization or filesystem health.

The complete data path is:

```
SD card -> SDIO1/card detect -> sd0 block device -> DFS/FatFs -> /sd
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

7.3 GD32H77D SDIO1 Features

The project enables GD32H77D SDIO1 and card detect. The controller handles commands, data lines, and block transfers. Active-low card detect indicates insertion only, not successful card or filesystem initialization.

7.4 RT-Thread SDIO, Block Device, and DFS Device Interface

After card discovery, the RT-Thread SDIO/MMCSd layer registers the sd0 block device. DFS/elm-FatFs accesses sectors and mounts the filesystem at /sd.

Primary device: sd0. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

7.5 Hardware

SDIO1 registers sd0 and mounts a FAT/FAT32 filesystem at /sd. Insert a usable SD card before testing.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

Insert a usable FAT/FAT32 SD card before testing the mount point.

7.6 Example

Source paths below are relative to the project directory in the SDK repository:

- ../../libraries/Board_Drivers/drv_filesystem.c
- ../../libraries/gd32_drivers/drv_sdio.c
- ../../libraries/gd32_drivers/config/sdio_config.h

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_sdcard
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- ls /sd

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

7.7 Runtime Results

Expected Behavior

- sd0 appears after card insertion.
- ls /sd lists FAT/FAT32 files and directories.
- Insertion and removal update device and mount status according to the configuration without disturbing the UART1 console.

8. Gino_driver_sdram

8.1 Introduction

This project is the GD32H77D Gino development board reference project for 32 MiB external SDRAM. It is used to learn, configure, and validate SDRAM Timing, Refresh, and Heap Extension separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

This project has no standalone primary device node.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks the primary device by default.
free	Print RT-Thread heap usage.	Confirm SDRAM has extended available heap.
sdram_test	Measure SDRAM read/write speed and verify data, showing elapsed time and MiB/s.	Allocate a 1 MiB buffer from the SDRAM heap, run 8 passes per operation, then free it.

8.2 SDRAM Timing, Refresh, and Heap Extension Details

The SDRAM controller must match bank, row/column, CAS, and refresh parameters. Once initialized, 32 MiB starts at 0xC0000000 and extends the heap. It has no standalone rt_device, so memory statistics and stress checks are used.

The complete data path is:

```
system clock -> EXMC SDRAM timing/refresh -> 0xC0000000 memory -> RT-Thread heap
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

8.3 GD32H77D EXMC and SDRAM Features

The GD32H77D external memory controller configures SDRAM bank, row/column, CAS latency, and refresh. The 32 MiB device starts at 0xC0000000 and is sensitive to timing and bus contention.

8.4 RT-Thread Memory Management Device Interface

SDRAM has no standalone rt_device. After BSP initialization it extends the heap, so applications use it through rt_malloc, rt_calloc, and rt_free and inspect total and remaining capacity with free.

This project has no standalone primary device node. Use list_device and gino_device_probe first to check the runtime state. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

8.5 Hardware

Initializes 32 MiB SDRAM at 0xC0000000 and extends the system heap; no standalone device node is expected.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

If display or camera features are later enabled, keep framebuffer/camera reservations out of the heap.

8.6 Example

Source paths below are relative to the project directory in the SDK repository:

- ../../libraries/Board_Drivers/drv_sdram_port.c
- ../../libraries/gd32_drivers/drv_sdram.c
- ../../libraries/Board_Drivers/board.h
- board/SConscript

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_sdram
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- free
- sdram_test

sdram_test takes no arguments and accesses only its allocated SDRAM heap buffer, preserving heap metadata, existing allocations, and display/camera reservations. It reports an error if allocation or SDRAM initialization has failed.

The output table shows sequential Write and Read speeds: Total KiB is the total data accessed per operation, Time ms is elapsed time, and MiB/s is throughput (1 MiB = 1024 * 1024 bytes). Each operation transfers 8 MiB in total. Data verification runs separately and prints Verify: PASS or the first failing address with its expected and actual values.

Measurements use the current CPU, cache, and SDRAM timing configuration. Write timing includes a D-Cache clean on every pass; read timing includes an invalidate on every pass to fetch data from SDRAM. Results include loop and scheduling overhead and represent effective CPU sequential throughput, not theoretical SDRAM bus bandwidth. Measurements shorter than one system tick show N/A.

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

8.7 Runtime Results

Expected Behavior

- The startup log reports SDRAM initialization.
- free shows substantially more heap than internal-RAM projects.
- sdram_test reports elapsed time and MiB/s for Write and Read, followed by Verify: PASS.
- Large allocate/write/verify/free cycles preserve data.

9. Gino_driver_spi

9.1 Introduction

This project is the GD32H77D Gino development board reference project for SPI3 bus. It is used to learn, configure, and validate SPI Synchronous Serial Protocol separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: spi3.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks spi3 by default.
spi_loopback [length] [count]	Run a full-duplex SPI3 loopback test and compare every received byte.	spi_loopback defaults to 32 bytes per round and 10 rounds; spi_loopback 256 100 runs 100 rounds of 256 bytes.
list_device	List registered system devices.	Confirm that UART, PIN, buses, and target devices exist.

9.2 SPI Synchronous Serial Protocol Details

SPI shifts data over SCK, MOSI, and MISO under master control, with a separate chip select per target. This project validates spi3 registration and external loopback transfers. The loopback command uses 8-bit data, mode 0, MSB first, a requested maximum clock of 1 MHz, and no chip select.

The complete data path is:

```
application -> RT-Thread SPI bus -> SPI3 -> SCK/MOSI/MISO -> external target
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

9.3 GD32H77D SPI3 Features

The project enables GD32H77D SPI3 with PE12/PF0/PF1 for clock and data. The controller provides programmable prescaling, CPOL/CPHA, and data width; chip select is managed by an attached SPI device or GPIO.

9.4 RT-Thread SPI Bus Device Interface

RT-Thread registers spi3 as a bus. A concrete target is attached with rt_spi_bus_attach_device or a BSP helper, then uses rt_spi_configure, rt_spi_transfer, and rt_spi_send/rt_spi_recv.

Primary device: spi3. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

9.5 Hardware

SPI3 uses PE12/PF0/PF1. Chip select, mode, and clock are configured by the target application.

For external loopback, disconnect external SPI targets and connect **PF1 (MOSI)** directly to **PF0 (MISO)**. **PE12 (SCK)** needs no jumper and can be observed with a logic analyzer. No CS connection is required. Use the configured pins instead if the SPI3 pin mapping has been changed.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

A registered idle SPI bus has no waveform until a concrete SPI device issues a transfer.

9.6 Example

Source paths below are relative to the project directory in the SDK repository:

- applications/main.c
- applications/device_probe.c
- ../../libraries/gd32_drivers/drv_spi.c
- ../../libraries/gd32_drivers/config/spi_config.h

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_spi
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- list_device
- spi_loopback: 32 bytes per round, 10 rounds.
- spi_loopback 256 100: 256 bytes per round, 100 rounds.
- spi_loopback --help: show parameter ranges and wiring.

length accepts 1-256 bytes and count accepts 1-1000 rounds, both in decimal. The command attaches a persistent spi3_loop device to spi3 on first use and reuses it on subsequent runs. Each round sends a changing byte pattern while receiving, checks the transfer length, and compares every byte. The current BSP uses polling transfers; the actual SCK frequency depends on the peripheral clock and prescaler.

Operation Steps

1. Disconnect external SPI targets and short PF1 (MOSI) to PF0 (MISO).
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run gino_device_probe, then spi_loopback. A successful default run prints SPI loopback PASS: 10 rounds, 320 bytes checked.
5. Run spi_loopback 256 100 for a longer test. On failure, check the transfer count or the reported round, zero-based byte offset, and TX/RX values against the wiring and pin configuration.

9.7 Runtime Results

Expected Behavior

Runtime logs use ulog with time, level, and tag: spi.demo for startup and spi.test for device probing and loopback tests. INFO/PASS is green, WARN for invalid arguments is yellow, and ERROR/FAIL is red in terminals that support ANSI colors. Command help uses plain ulog output.

- list_device shows spi3.
- gino_device_probe finds the SPI3 bus.
- spi_loopback produces clock/data activity and prints PASS when the received bytes match the transmitted bytes.
- Missing or incorrect loopback wiring normally produces FAIL with the first mismatching byte. Registration alone does not validate data transfer. The driver's polling waits currently have no timeout, so a controller/clock fault can block the command.

10. Gino_driver_usb_device

10.1 Introduction

This project is the GD32H77D Gino development board reference project for USBHS0 device. It is used to learn, configure, and validate USB Device Enumeration and CDC separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: usbd.

CDC data device: vcom. After the PC opens the USB COM port with DTR enabled, the application sends a numbered test message every second and echoes received bytes.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks usbd by default.
list_device	List registered system devices.	Confirm that UART, PIN, buses, and target devices exist.

10.2 USB Device Enumeration and CDC Details

USBHS0 acts as a device, responding to reset and enumeration with device, configuration, interface, and endpoint descriptors. The configuration enables the RT-Thread USB device stack and CDC composite support. The PC loads the corresponding driver based on these descriptors.

The complete data path is:

```
PC host -> Type-C D+/D- -> USBHS0 device -> RT-Thread USB device/CDC ->
  application endpoints
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

10.3 GD32H77D USBHS0 Device Features

GD32H77D USBHS0 is configured as a device on the Type-C PA11/PA12 interface. It responds to host reset and endpoint transactions; the RT-Thread configuration enables device, composite, and CDC support.

10.4 RT-Thread USB Device Interface

The RT-Thread USB device stack defines devices, configurations, interfaces, and endpoints through descriptors and dispatches class requests to CDC. It is not a direct replacement for a plain character rt_device; applications normally exchange data through USB class callbacks or class-specific interfaces.

The controller is registered as usbd, and the CDC serial interface is registered as vcom. The application opens vcom with rt_device_open(), reads and echoes data with rt_device_read() / rt_device_write(), and checks RT_USBD_CLASS_CTRL_CONNECTED before sending. This state follows the DTR signal set by the PC serial tool. rt_kprintf() and FinSH/MSH continue to use uart1.

10.5 Hardware

USBHS0 device uses the Type-C connector and PA11/PA12. Use a data-capable USB cable.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

Use a data-capable cable and confirm the PC actually enumerates the device instead of only powering the board.

10.6 Example

Source paths below are relative to the project directory in the SDK repository:

- ../../libraries/gd32_drivers/usb/drv_otghs_dev.c
- ../../libraries/gd32_drivers/usb/drv_usb_dev.c
- ../../libraries/gd32_drivers/usb/gd32h7_usb_hw.c
- rtconfig.h

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_usb_device
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- list_device

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device on UART1 and confirm that both usbd and vcom are registered. gino_device_probe checks the usbd controller.
4. Connect the USBHS0 Type-C port to the PC, open the enumerated USB COM port at 115200-8-N-1, and enable DTR in the serial tool.
5. Check for one numbered test message per second. Send a short string such as hello\r\n and confirm that it is echoed; disable local echo in the PC tool to avoid mistaking local input for a device response.
6. Close and reopen the USB COM port with DTR enabled, then confirm that periodic messages resume. This is a low-rate communication example; the receive buffer is 128 bytes, so use short packets for the echo test.

10.7 Runtime Results

Expected Behavior

- A data-capable cable lets the PC enumerate a USB/CDC device.
- The device does not repeatedly connect/disconnect or remain unknown.
- UART1 remains available for enumeration logs.
- With the USB COM port open and DTR enabled, one message is received per second:

```
Gino USB CDC test: 0
Gino USB CDC test: 1
Gino USB CDC test: 2
```

- Received bytes are echoed unchanged. Periodic messages share the same COM port with echo data.
- Transmission pauses while DTR is cleared or USB is disconnected and resumes after reopening the port with DTR enabled.

11. Gino_driver_usb_host

11.1 Introduction

This project is the GD32H77D Gino development board reference project for USBHS1 host and mass-storage. It is used to learn, configure, and validate USB Host Enumeration and Mass Storage separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: usbh.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks usbh by default.
list_device	List registered system devices.	Confirm that UART, PIN, buses, and target devices exist.
ls /udisk	List files on the mounted USB mass-storage filesystem.	Requires an enumerated FAT/FAT32 drive.

11.2 USB Host Enumeration and Mass Storage Details

USBHS1 acts as host and PF2 controls downstream VBUS. The host stack detects, resets, reads descriptors, and selects a class driver. Mass storage then becomes a block device mounted by DFS/FatFs at /udisk.

The complete data path is:

```
PF2 VBUS -> USB-A drive -> USBHS1 enumeration -> mass storage -> block device ->
  FatFs -> /udisk
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

11.3 GD32H77D USBHS1 Host Features

GD32H77D USBHS1 operates as host, with PF2 controlling USB-A VBUS. The controller handles reset, control transfers, and pipes while the RT-Thread host stack parses descriptors and selects class drivers.

11.4 RT-Thread USB Host and DFS Device Interface

The mass-storage class registers a block device, which DFS/FatFs mounts at /udisk. Applications use DFS/POSIX file APIs instead of raw USB pipes.

Primary device: usbh. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

11.5 Hardware

USBHS1 host uses the USB-A connector, with VBUS controlled by PF2; compatible drives mount at /udisk.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

Measure VBUS on the USB-A connector first if no device is enumerated.

11.6 Example

Source paths below are relative to the project directory in the SDK repository:

- ../../libraries/gd32_drivers/usb/drv_otghs_host.c
- ../../libraries/gd32_drivers/usb/drv_usb_host.c
- ../../libraries/gd32_drivers/usb/gd32h7_usb_hw.c
- ../../libraries/Board_Drivers/drv_filesystem.c

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_driver_usb_host
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe
- list_device
- ls /udisk

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

11.7 Runtime Results

Expected Behavior

- A compatible drive produces enumeration and mass-storage logs.
- Host/storage devices appear in list_device.
- ls /udisk lists FAT/FAT32 files.

Display and Camera

1. Gino_display_camera

1.1 Introduction

This example provides OV7670 camera capture and live preview through LVGL and the MIPI LCD on the GD32H77D Gino board. Preview starts automatically after LVGL initialization.

The primary device is ov7670. UART1 is the FinSH/MSH console at 115200-8-N-1, and PC4 is the run indicator.

1.2 Capture and Display

I2C2/SCCB configures OV7670, and TIMER0 supplies XCLK on PE9. DCI receives the parallel pixel stream. DMA1 channel 7 writes a dedicated capture buffer in internal AXI SRAM. After cache maintenance and RGB565 processing, rt_device_read copies a QVGA frame into the application's destination buffer.

The preview thread uses two additional internal AXI SRAM buffers. It tracks buffers being written, ready, and displayed so the LVGL image source remains stable until rendering completes. LVGL renders through a 32-line draw buffer, IPA updates the back framebuffer in SDRAM, and the LCD driver swaps framebuffers during vertical blanking.

1.3 Memory Requirements

The DMA capture buffer and the two preview buffers are aligned to 32 bytes. LCD framebuffers use the reserved SDRAM area; with LCD enabled, the SDRAM heap starts at 0xC0300000.

Board initialization completes SDRAM and MPU setup before enabling D-Cache. LCD use requires successful SDRAM initialization. A bootloader must clean and disable D-Cache before handing control to this firmware.

1.4 Device Interfaces

The ov7670 device supports rt_device_open and rt_device_read. Allocate an output buffer large enough for a 320 x 240 RGB565 frame; a successful read returns the frame size in bytes.

The preview uses the lcd graphic device, the LVGL image API, and the LVGL thread for UI updates. GT911 supplies touch input when enabled.

1.5 Hardware

- OV7670: power, common ground, reset, XCLK, PCLK, VSYNC, HREF, D0-D7, and I2C2/SCCB connections.
- Camera control: I2C2 on PA8/PC9; XCLK on PE9.
- Display: MIPI LCD with SDRAM; GT911 touch on I2C3.
- Console: UART1 on PA2/PA3, AF7, 115200-8-N-1.
- Run LED: PC4, toggled every 500 ms.

Connect the camera with the board powered off and use the module's required supply and signal levels.

1.6 Example Operation

Source paths below are relative to the project directory in the SDK repository:

- applications/main.c
- applications/lv_port.c
- applications/ov7670_preview.c
- ../../libraries/Board_Drivers/drv_ov7670.c
- ../../libraries/Board_Drivers/ports/camera/sensors/ov7670.c

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_display_camera
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories. This project also links the bundled packages: LVGL-latest, gt911-latest.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands and Operation Steps

Command	Function
ov7670_preview start	request live preview on the LCD
ov7670_preview stop	stop live preview and release the preview screen

1. Connect the camera, LCD, power, and console.
2. Build and download this project using RT-Thread Studio or MDK5.

3. Reset the board and wait for the LCD to display the camera image automatically.
4. Use the start and stop commands when manual preview control is needed.

1.7 Expected Behavior

- LVGL initializes the display and starts OV7670 preview.
- The LCD displays continuous QVGA RGB565 camera images.
- The preview can be stopped and restarted from the console.

2. Gino_display_gd32embeddedui

A standalone GD32H77D Gino example for GD32EmbeddedUI exports, using **LVGL 8.3.11**, a **720×720 RGB565 MIPI DSI LCD** and **GT911 touch**.

2.1 Editable UI package

The shared *GD32EmbeddedUI-demos package* contains the editable *GD32Embeddedui.gdlvgl* project, resources, generated code and custom application code. Open that file in GD32EmbeddedUI, edit the UI and generate code in place. The board project uses these same sources through its packages/GD32EmbeddedUI-demos directory link.

The supplied design is empty, with custom_init() reserved for application initialization. Add a label or button in the designer and generate code to check display output and touch.

2.2 Hardware and initialization

The example reuses the LVGL 8 board configuration, LCD, GT911 and SDRAM drivers. Two 200-line RGB565 buffers in AXI SRAM feed IPA transfers to the SDRAM display framebuffer. The RT-Thread LVGL thread initializes LVGL, display and input, calls ui_init(), then handles rendering and input continuously.

The generated entry already calls custom_init(). Do not initialize LVGL or call these entry points again from main(). Put application logic in custom/ and synchronize any UI access from other threads with the LVGL thread.

2.3 Open the board project

From this directory:

```
.\mklinks.bat
scons --target=mdk5
```

Open project.uvprojx in MDK5. The target command generates project files only. For RT-Thread Studio, import this directory or create the example through the SDK catalog. Update Studio source lists using scons --target=eclipse and refresh the project. On Linux, establish links with sh mklinks.sh.

Shared dependencies are rt-thread, libraries, LVGL-v8.3.11, gt911-latest and GD32EmbeddedUI-demos. SDK exports copy the dependencies into the standalone project. In that case, edit its local packages/GD32EmbeddedUI-demos/GD32Embeddedui.gdlvgl.

2.4 Regenerate the UI

Keep the designer at LVGL 8.3.11, 720×720 and 16-bit color. Export images and fonts as C arrays/ static fonts under generated/. SCons recursively collects C sources and include paths from generated/ and custom/. After adding or removing files, regenerate the MDK/Studio project before building firmware.

applications/lv_conf.h includes the exported ui_config.h and lv_conf_patch.h so configuration applies to LVGL and the generated code. Windows simulator sources and third-party libraries are excluded from firmware. This example has no resource filesystem, FreeType or FFmpeg integration; raw files in assets/ and PC media paths require additional board support.

2.5 Board checks

- UART1 prints the project name; the PC4 LED blinks.
- Use list_device and gino_device_probe to inspect lcd and gt911.
- Successful display initialization is followed by GD32EmbeddedUI initialized (720x720, RGB565).
- The empty design has no controls. Add a button in the designer, generate code and verify touch on the board.

Validation covers project generation and relevant source syntax checks. Full firmware build, flashing and hardware operation remain to be verified.

3. Gino_display_lvgl

3.1 Introduction

This project uses **LVGL 9.x** from ../../packages/LVGL-latest (currently the bundled 9.6 development version). For a separate fixed LVGL 8.3.11 project, use Gino_display_lvgl8.

This project is the GD32H77D Gino development board reference project for MIPI DSI LCD, GT911 and LVGL graphics. It is used to learn, configure, and validate LVGL Rendering, Input, and Double Buffering separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: lcd.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks lcd by default.

3.2 LVGL Rendering, Input, and Double Buffering Details

A dedicated LVGL thread initializes objects, display, and touch and periodically calls lv_timer_handler to process animations, input, and refreshes. This project uses two 200-line partial draw buffers in internal AXI SRAM. IPA asynchronously copies rendered rectangles to a single scanout framebuffer in SDRAM and releases each draw buffer after transfer completion.

The complete data path is:

```
GT911 input -> LVGL objects/events -> RGB565 buffer -> cache clean/IPA -> TLI/DSI
  -> LCD
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

3.3 GD32H77D SDRAM, TLI/DSI, and I2C3 Features

The project combines SDRAM, TLI, DSI, IPA, I2C3, and timer PWM. TLI/DSI scan framebuffers, GT911 supplies touch, IPA accelerates copies, and M7 D-Cache must be cleaned before hardware reads.

Board initialization protects unused external address windows with MPU region 0; higher-priority regions keep valid SDRAM accessible. Mapped OSPI requires separate accessible MPU regions before this guard can be used.

D-Cache remains disabled until SDRAM initialization and display-memory MPU setup finish. The LCD uses the MPU and cache attributes configured during board initialization. A bootloader must clean and disable D-Cache before handing control to this firmware.

3.4 RT-Thread LVGL, Graphic, and Touch Device Interface

The RT-Thread LVGL port creates an LVGL thread that calls lv_timer_handler. It uses the lcd graphic and gt911 touch devices. LVGL 9 registers buffers through lv_display_set_buffers; LVGL 8 uses lv_disp_draw_buf_init and lv_disp_drv_register. The flush callback releases draw buffers after hardware finishes using them.

Primary device: lcd. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

LVGL 8 and 9 Compatibility

Gino_display_lvgl, Gino_display_lvgl8, Gino_display_camera, and Gino_factory select their APIs automatically through LVGL_VERSION_MAJOR. The shared adapter is ../../libraries/Board_Drivers/lvgl_compat.h. It covers display flush and input callback types, image and widget API names; each project's lv_conf.h configures native RGB565.

Run mklinks.bat once when using the project inside the SDK repository (sh mklinks.sh on Linux). This links RT-Thread, the shared board libraries, GT911, and LVGL-latest into the project. The Studio SDK manifest also includes the matching offline package when creating a standalone project. packages/SConscript selects only the LVGL version configured in .config and rtconfig.h, so stale downloads are excluded. Regenerate MDK/Studio metadata after intentionally changing package versions. LVGL_VERSION_MAJOR is supplied by LVGL itself and must not be overridden manually.

LVGL 8.3.11, 8.4.0, and the bundled 9.6 development version passed syntax checks of the affected sources. LVGL 8 buffer sizes are pixel counts, while LVGL 9 uses bytes. When camera support is disabled and the camera or factory project uses two scanout framebuffers, LVGL 8 renders full frames before vertical blank swaps; LVGL 9 uses direct rendering. Refresh rate and touch behavior require validation on the board.

3.5 Hardware

Combines the LCD, GT911, SDRAM, and the offline LVGL package.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

Interact with the screen and check that animations and touch responses remain stable.

3.6 Example

Source paths below are relative to the project directory in the SDK repository:

- applications/lv_port.c
- applications/lv_conf.h
- ../../libraries/Board_Drivers/drv_lcd.c
- ../../libraries/Board_Drivers/lvgl_compat.h
- ../../packages/LVGL-latest/env_support/rt-thread/lv_rt_thread_port.c

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_display_lvgl
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories. This project also links the bundled packages: LVGL-latest, gt911-latest.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

3.7 Runtime Results

Expected Behavior

- The LVGL demo appears and touch operates controls.
- Draw buffers alternate while IPA transfers screen updates to the LCD.

4. Gino_display_lvgl8

4.1 Introduction

This project is fixed to **LVGL 8.3.11**. Its offline sources are in ../../packages/LVGL-v8.3.11. The LVGL 9.x example is Gino_display_lvgl.

This project is the GD32H77D Gino development board reference project for MIPI DSI LCD, GT911 and LVGL graphics. It is used to learn, configure, and validate LVGL Rendering, Input, and Double Buffering separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: lcd.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
gino_device_probe	Check whether the primary device used by this example is registered.	Checks lcd by default.

4.2 LVGL Rendering, Input, and Double Buffering Details

A dedicated LVGL thread initializes objects, display, and touch and periodically calls lv_timer_handler to process animations, input, and refreshes. This project uses two 200-line partial draw buffers in internal AXI SRAM. IPA asynchronously copies rendered rectangles to a single scanout framebuffer in SDRAM and releases each draw buffer after transfer completion.

The complete data path is:

```
GT911 input -> LVGL objects/events -> RGB565 buffer -> cache clean/IPA -> TLI/DSI
  -> LCD
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

4.3 GD32H77D SDRAM, TLI/DSI, and I2C3 Features

The project combines SDRAM, TLI, DSI, IPA, I2C3, and timer PWM. TLI/DSI scan framebuffers, GT911 supplies touch, IPA accelerates copies, and M7 D-Cache must be cleaned before hardware reads.

Board initialization protects unused external address windows with MPU region 0; higher-priority regions keep valid SDRAM accessible. Mapped OSPI requires separate accessible MPU regions before this guard can be used.

D-Cache remains disabled until SDRAM initialization and display-memory MPU setup finish. The LCD uses the MPU and cache attributes configured during board initialization. A bootloader must clean and disable D-Cache before handing control to this firmware.

4.4 RT-Thread LVGL, Graphic, and Touch Device Interface

The RT-Thread LVGL port creates an LVGL thread that calls lv_timer_handler. It uses the lcd graphic and gt911 touch devices. LVGL 9 registers buffers through lv_display_set_buffers; LVGL 8 uses lv_disp_draw_buf_init and lv_disp_drv_register. The flush callback releases draw buffers after hardware finishes using them.

Primary device: lcd. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

LVGL 8 and 9 Compatibility

Gino_display_lvgl, Gino_display_lvgl8, Gino_display_camera, and Gino_factory select their APIs automatically through LVGL_VERSION_MAJOR. The shared adapter is ../../libraries/Board_Drivers/lvgl_compat.h. It covers display flush and input callback types, image and widget API names; each project's lv_conf.h configures native RGB565.

Run mklinks.bat once when using the project inside the SDK repository (sh mklinks.sh on Linux). This links RT-Thread, the shared board libraries, GT911, and LVGL-v8.3.11 into the project. The Studio SDK manifest also includes the matching offline package when creating a standalone project. packages/SConscript selects only the LVGL version configured in .config and rtconfig.h, so stale downloads are excluded. This project also checks the source version at compile time and requires 8.3.11. Use Gino_display_lvgl for 9.x. LVGL_VERSION_MAJOR is supplied by LVGL itself and must not be overridden manually.

LVGL 8.3.11, 8.4.0, and the bundled 9.6 development version passed syntax checks of the affected sources. LVGL 8 buffer sizes are pixel counts, while LVGL 9 uses bytes. When camera support is disabled and the camera or factory project uses two scanout framebuffers, LVGL 8 renders full frames before vertical blank swaps; LVGL 9 uses direct rendering. Refresh rate and touch behavior require validation on the board.

4.5 Hardware

Combines the LCD, GT911, SDRAM, and the offline LVGL package.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

Interact with the screen and check that animations and touch responses remain stable.

4.6 Example

Source paths below are relative to the project directory in the SDK repository:

- applications/lv_port.c
- applications/lv_conf.h
- ../../libraries/Board_Drivers/drv_lcd.c
- ../../libraries/Board_Drivers/lvgl_compat.h
- ../../packages/LVGL-v8.3.11/env_support/rt-thread/lv_rt_thread_port.c

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_display_lvgl8
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories. This project also links the bundled packages: LVGL-v8.3.11, gt911-latest.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- gino_device_probe

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

4.7 Runtime Results

Expected Behavior

- The LVGL demo appears and touch operates controls.
- Draw buffers alternate while IPA transfers screen updates to the LCD.

Components

1. Gino_component_mqtt

1.1 Introduction

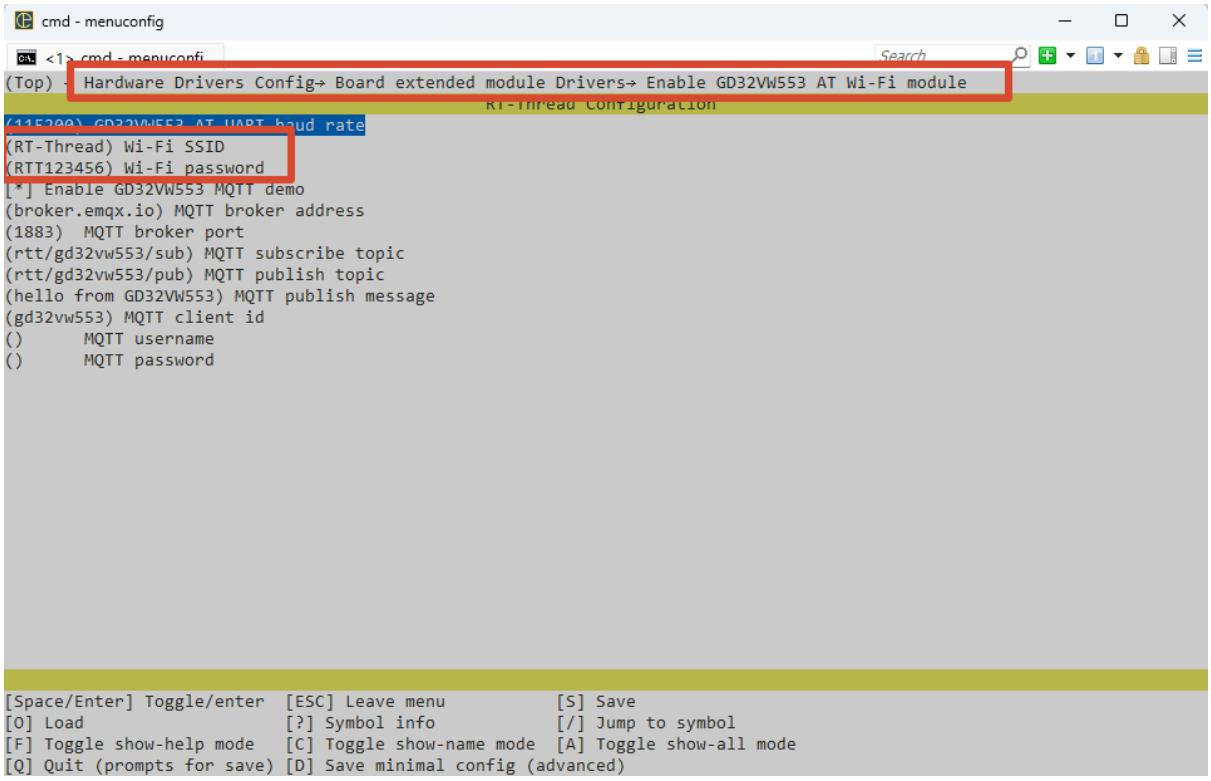
This project is the GD32H77D Gino development board reference project for GD32VW553, SAL, and kawaii-mqtt publish/subscribe. It is used to learn, configure, and validate MQTT publish/subscribe separately. The standalone project enables only the drivers and components required by this feature, and can be used as a reference for application development and integration.

Primary device: wifi0.

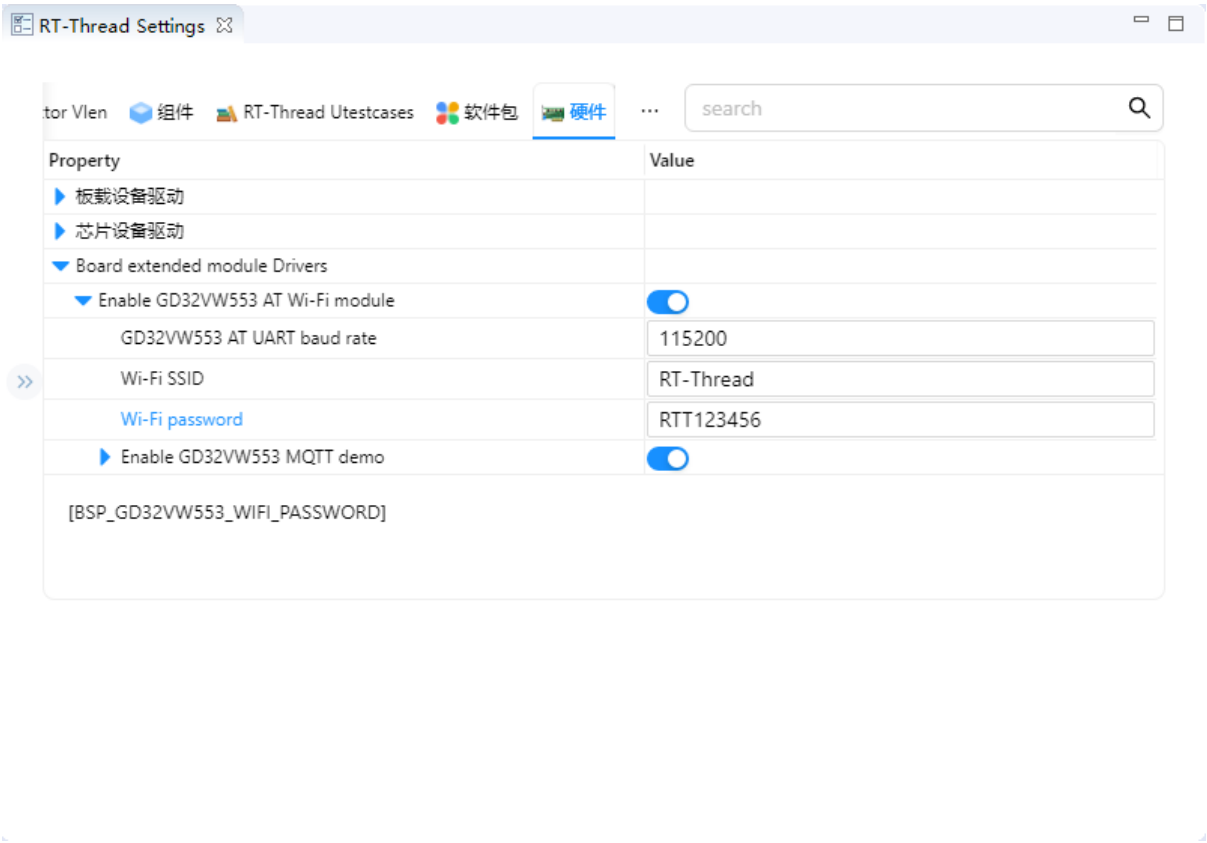
All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Before running the example, change the Wi-Fi settings to match your own network.

Configuration in Env:



Configuration in RT-Thread Studio:



Current project provides the following MSH commands:

Command	Function	Example/Note
gd32vw553_mqtt_start	Start the GD32VW553 MQTT demo, wait for wifi0 to connect, then connect to the Broker and subscribe to the topic.	gd32vw553_mqtt_start
gd32vw553_mqtt_pub <topic> <message>	Publish one message through the current MQTT connection.	gd32vw553_mqtt_pub rtt/gino hello
gd32vw553_mqtt_stop	Stop the MQTT demo, disconnect MQTT/TCP, and release client resources.	gd32vw553_mqtt_stop
ifconfig	Check the connection state and IP address of wifi0.	Confirm that the network is ready.
list_device	List registered system devices.	Confirm that UART, PIN, netdev, and other dependencies exist.
gino_device_probe	Check whether the primary device used by this example is registered.	Checks wifi0 by default.

You can use your own topics when running the example.

1.2 MQTT Publish/Subscribe Protocol Details

The demo first waits for wifi0 to become ready and temporarily selects it as the SAL default netdev. kawaii-mqtt handles MQTT encoding/decoding and session state. The underlying socket goes through SAL, AT socket, UART4, and GD32VW553. The publish command directly calls the active MQTT client and returns the real send result to MSH.

The complete data path is:

```
MSH publish -> kawaii-mqtt -> socket/SAL -> AT socket -> UART4 -> GD32VW553 ->
  Broker
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

1.3 GD32H77D UART4 and Cortex-M7 Features

The GD32H77D side mainly uses Cortex-M7, UART4, RT-Thread IPC, and memory resources. Wi-Fi/TCP/IP is offloaded to GD32VW553. MQTT packets are still encoded and decoded by kawaii-mqtt on the H7 side, then sent through SAL/AT socket.

1.4 RT-Thread AT Device, SAL, and MQTT Device Interface

The network foundation is the wifi0 at_device/netdev and SAL socket. The demo protects client state with an RT-Thread mutex, then calls mqtt_connect, mqtt_subscribe, and mqtt_publish.

Primary device: wifi0. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

1.5 Hardware

GD32VW553 communicates through UART4 (PB12/PB13), with PC5 controlling reset. Configure Wi-Fi, Broker, topics, and credentials locally.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

1.6 Example

Source paths below are relative to the project directory in the SDK repository:

- applications/gd32vw553_mqtt_demo.c
- ../../packages/at_device-latest/samples/at_sample_gd32vw553.c
- ../../packages/kawaii-mqtt-latest/
- ../../packages/at_device-latest/class/gd32vw553/

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_component_mqtt
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories. This project also links the bundled packages: at_device-latest, kawaii-mqtt-latest.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- ifconfig
- gd32vw553_mqtt_start
- gd32vw553_mqtt_pub rtt/gino hello
- gd32vw553_mqtt_stop

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

1.7 Runtime Results

Expected Behavior

- ifconfig shows that wifi0 is connected.
- The start command completes TCP/MQTT connection and subscribes to the target topic.
- After publishing, the Broker/subscriber receives the payload, the receive callback prints subscribed messages, and the stop command releases the client and restores the default netdev.

Factory Demo

1. Gino_factory

1.1 Introduction

This project combines LVGL, camera, MQTT, Ethernet, storage, and common peripherals on the GD32H77D Gino board. Use the touch interface and console to operate the enabled features.

Primary device: lcd.

All examples keep uart1 as the FinSH/MSH console at 115200-8-N-1, and PC4 LED is used as the run indicator.

Current project provides the following MSH commands:

Command	Function	Example/Note
ov7670_preview start	Request live camera preview on the LCD.	Use after the display and camera are ready.
gd32vw553_mqtt_start	Start the GD32VW553 MQTT demo.	Connects Wi-Fi, Broker, and subscription path.
ifconfig	Check the connection state and IP address of the netdev.	Confirm that the network is ready.
list_device	List registered system devices.	Confirm that UART, PIN, buses, and target devices exist.
i2c scan hwi2c1 08 78	Scan external I2C addresses 0x08-0x77.	Provided by i2c-tools; the external bus is hwi2c1.

The I2C touch page scans only the external hwi2c1 bus (SCL PH4, SDA PB11). Its Scan button uses address-only transfers in board_demo_backend.c. The camera and GT911 touch controller use hwi2c2 and hwi2c3 respectively; these buses have no scan entry in the UI. The console uses the bundled i2c-tools v1.0.0 package in hardware bus mode. Run mklinks.bat on Windows or sh mklinks.sh on Linux before opening the project. Run i2c for scan/read/write usage; scan ranges are hexadecimal with an exclusive stop address.

The SPI page performs a full-duplex SPI3 loopback test: connect PF1 (MOSI) to PF0 (MISO); SCK is PE12. The test uses mode 0, 8-bit data and no chip select. Select the maximum clock frequency and transfer length, then press Run loopback. The screen log shows queuing, configuration, the complete TX/RX hex data and PASS/FAIL, including the first mismatching byte offset and values.

The I2C page uses a 16-column address matrix like i2c-tools: responding devices show their address, -- means no response, .. means unscanned, and !! marks a probe error. Reserved addresses remain blank. The scan covers 0x08-0x77, updates each address result, and reports the device count and elapsed time.

There is no per-address sleep. Bus locking waits at most 100 ms; while holding the lock, each probe temporarily caps the driver's wait-stage timeout at 20 ms and then restores it. Timeouts or other bus errors stop the scan and preserve partial results instead of reporting an incomplete scan as an empty bus.

The Scan log retains responding addresses, errors and the final result. SPI and I2C tests disable their buttons and parameter selectors while pending or running, and report submission errors. Logs also go to the console; each test retains the latest 3 KB for the current request in a scrollable screen panel. These screen logs cover requests made from the touch UI. The I2C page scans addresses only; it does not program chip data or firmware.

Wi-Fi Configuration

Before running the factory demo, configure the Wi-Fi SSID and password for your own network. The procedure and screenshots follow Gino_component_mqtt, but each project has its own configuration: make these changes in Gino_factory.

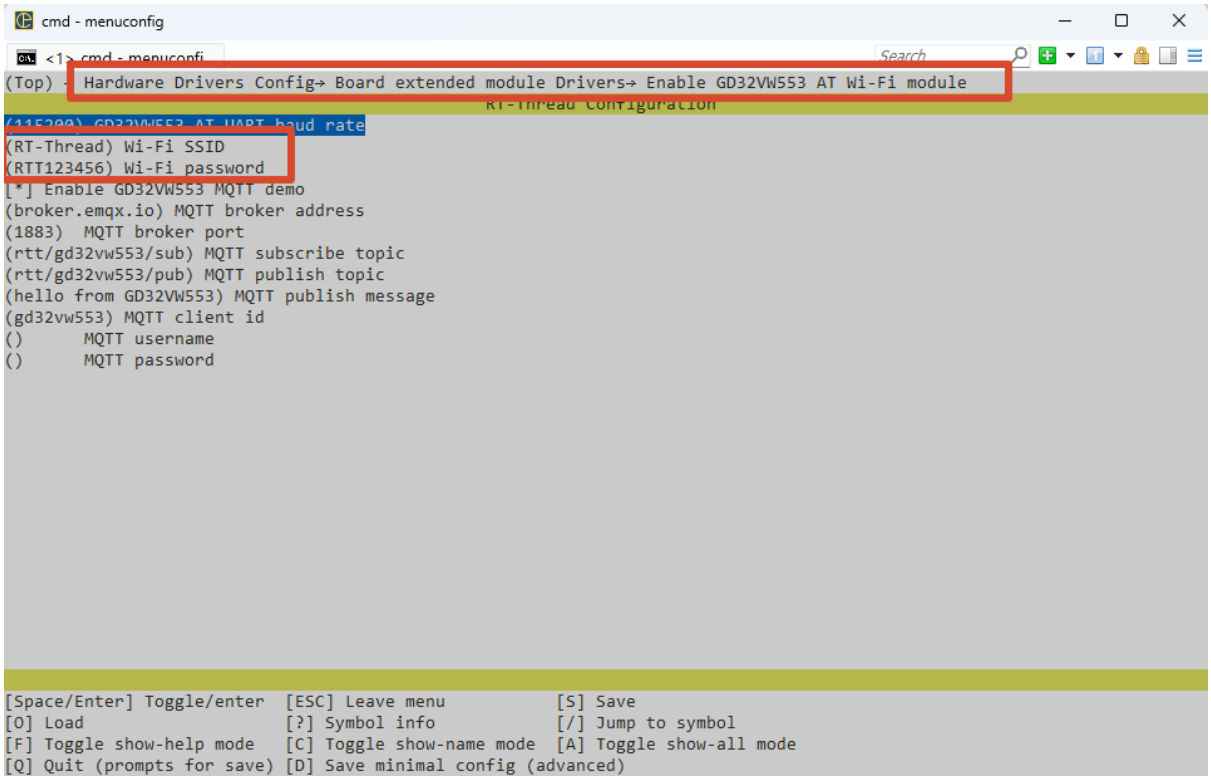
The onboard GD32VW553 AT module uses UART4 (TX PB12, RX PB13, AF14), with PC5 controlling reset. Its network device is wifi0; UART1 remains the console.

Env

1. In Env, enter projects\Gino_factory from the SDK root and run mklinks.bat if the shared directory links have not been created.
2. Run menuconfig and open the following menu:

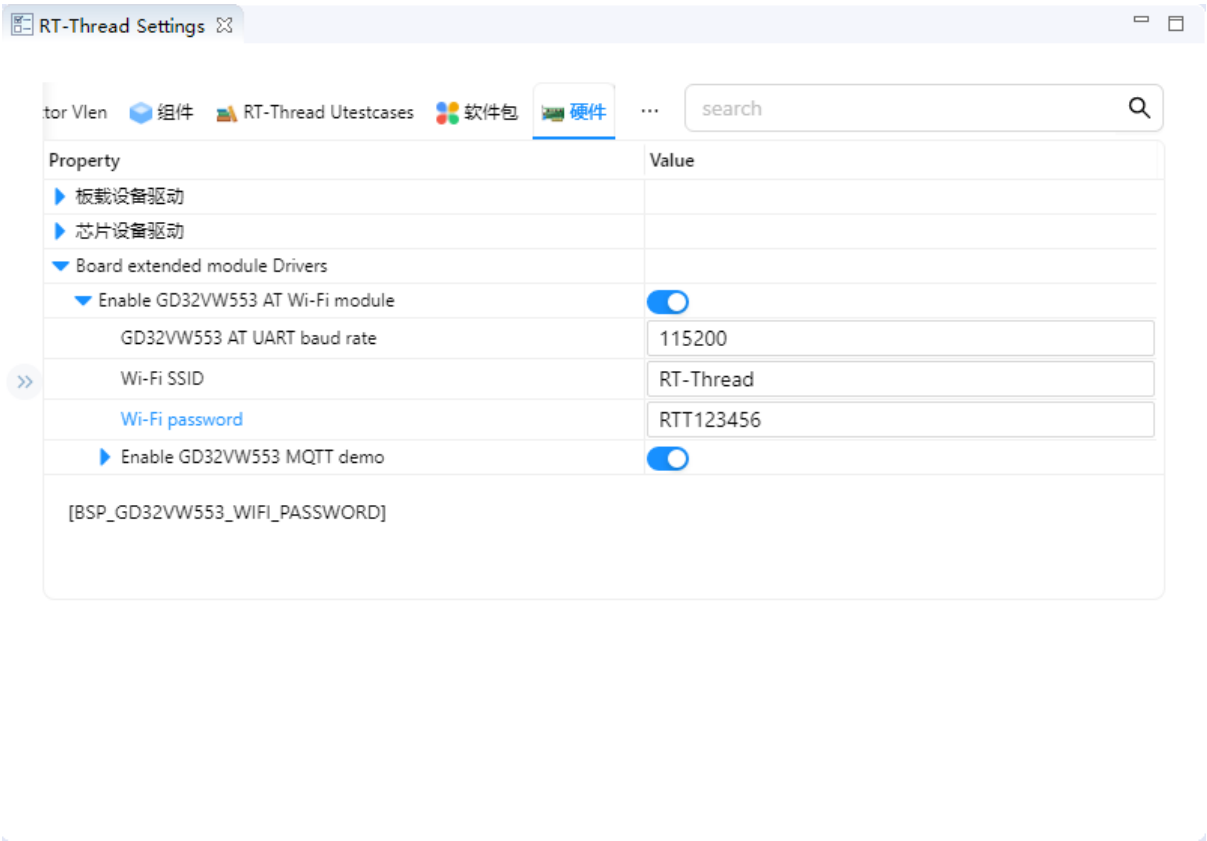
```
Hardware Drivers Config
    -> Board extended module Drivers
        -> Enable GD32VW553 AT Wi-Fi module
```

3. Keep the module enabled. Open RT-Thread online packages -> IoT - internet of things -> AT DEVICE, select latest, and configure GigaDevice GD32VW553 -> Enable sample. Set WIFI ssid and WIFI password to your network's values. Keep AT UART baud rate at 115200 unless the module firmware uses a different rate. The board enables the registration sample automatically and defaults its reset pin to PC5 (37), active low.
4. Save and exit, then run scons --pyconfig-silent to update rtconfig.h. For MDK5, also run scons --target=mdk5 to update the project files.



RT-Thread Studio

1. Open the factory project’ s RT-Thread Settings and select the hardware tab.
2. Expand Board extended module Drivers -> Enable GD32VW553 AT Wi-Fi module and keep the module enabled.
3. Switch to the software package settings. Under AT DEVICE (latest) -> GigaDevice GD32VW553 -> Enable sample, set WIFI ssid and WIFI password, confirm the AT UART baud rate, and save. The screenshots show the previous configuration layout; Wi-Fi settings now belong to the package sample.



The two fields correspond to GD32VW553_SAMPLE_WIFI_SSID and GD32VW553_SAMPLE_WIFI_PASSWORD. Use your own network values instead of the values shown in the screenshots. After saving, rebuild and download the factory firmware, then reset the board to apply the configuration.

Wi-Fi and MQTT Verification

Run ifconfig on UART1 and confirm that wifi0 is up, its link is up, and its IP address is not 0.0.0.0. The factory project also enables Ethernet e0; an Ethernet connection alone does not confirm that Wi-Fi is ready. If wifi0 does not connect, check the SSID, password, access point availability, module power/reset, and AT UART baud rate.

To use MQTT, keep Enable GD32VW553 MQTT demo enabled in the board GD32VW553 configuration menu and set the broker address/port, subscribe/publish topics, client ID, and optional username/password for your service. You can use your own topics. Use a numeric IPv4 broker address when the module firmware does not support DNS through AT+CIPDOMAIN.

After applying the configuration and confirming that wifi0 is ready, run:

```
gd32vw553_mqtt_start
gd32vw553_mqtt_pub rtt/gino hello
gd32vw553_mqtt_stop
```

Wait for the MQTT connection and subscription to succeed before publishing. Replace rtt/gino with your own topic and confirm that the broker/subscriber receives the message. The MQTT demo temporarily selects wifi0 as the default network device and restores the previous default when it stops.

1.2 Concurrent Peripheral System Integration Details

The factory project verifies that independently working features can share pins, initialization levels, DMA, cache, SDRAM, default routes, and heap. Display continuously scans SDRAM, camera and ENET use DMA, Wi-Fi/MQTT selects wifi0 through SAL, and filesystems mount after devices register. This more closely represents a real product workload.

The complete data path is:

```
driver registration -> DFS/lwIP/LVGL/SAL -> concurrent display + camera + storage
  + e0/wifi0 + MQTT
```

The protocol layer only defines communication and data processing rules. Final validation still depends on controller clocks, pin multiplexing, interrupt/DMA handling, and upper-layer state machines. Device discovery, bus registration, or a successful build cannot replace a complete data transfer test.

1.3 GD32H77D Multiple Buses, DMA Engines, and Memories Features

The integrated project uses SDRAM/TLI/DSI, DCI/DMA, ENET1, OSPI0, SDIO1, USBHS, CAN, SPI, I2C, UART, and timers together. It validates shared pins, DMA, cache, bandwidth, memory regions, and interrupt priorities.

1.4 RT-Thread Integrated Devices and Components Device Interface

RT-Thread runs the device framework, DFS/FAL/FatFs, lwIP/netdev/SAL, at_device, LVGL, touch, camera, and MQTT threads together. Stable names, init levels, and IPC coordinate modules.

Primary device: lcd. Use list_device and gino_device_probe first to check whether the device has been registered. Upper-layer file system, network, or GUI components still need separate validation for mount, link, or refresh state.

1.5 Hardware

Enables the 720 x 720 LCD, GT911, OV7670, GD32VW553, ENET1, OSPI flash, SDIO, USB, CAN, SPI, I2C, and PWM. Follow each standalone project’ s wiring and electrical requirements.

Default console: UART1, PA2/PA3, AF7, 115200-8-N-1.

1.6 Example

Source paths below are relative to the project directory in the SDK repository:

- applications/board_demo.c
- applications/board_demo_backend.c
- applications/lv_port.c
- applications/ov7670_preview.c
- applications/gd32vw553_mqtt_demo.c
- board/SConscript

tests/run_host_tests.ps1 uses host GCC and a simulated bus to check command dispatch, scan results, timeout aborts and timeout restoration without building firmware. It defaults to MSYS2 UCRT64 GCC; use -Compiler to select another path. Tests extract the current backend functions directly and do not replace board checks of wiring, ACK signals or actual scan timing.

Read applications/main.c and applications/device_probe.c first, then follow the data path into the corresponding driver, component, or package. The example keeps MSH commands so device registration and runtime state can be observed without changing application code.

Generate the MDK5 Project

In a PowerShell session provided by Env 2.0 or later, run the following from the SDK root:

```
cd projects\Gino_factory
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat creates links to the shared rt-thread and libraries directories. This project also links the bundled packages: LVGL-latest, gt911-latest, at_device-latest, kawaii-mqtt-latest, i2c-tools-v1.0.0.

This generates project.uvprojx in the current directory. After changing feature configuration or source selection, run scons --target=mdk5 again to update the project.

Runtime Commands

- ov7670_preview start
- gd32vw553_mqtt_start
- ifconfig
- list_device
- i2c scan hwi2c1 08 78

Operation Steps

1. Check power, wiring, external modules, and interface logic levels.
2. Reset the development board and confirm that the UART1 console is available and PC4 LED blinks normally.
3. Run list_device and confirm that dependency buses and target devices are registered.
4. Run the commands above in order while observing return values, external waveforms, network state, or display results.

1.7 Runtime Results

Expected Behavior

- list_device shows enabled display, touch, camera, storage, network, and bus devices.
- LVGL, preview, Ethernet/Wi-Fi, and filesystems start independently and coexist under reasonable load.
- The touch UI provides board feature controls; the console can operate camera, networking, and MQTT.
- The I2C touch page and i2c scan list responding slave addresses on the selected bus.
- The SPI loopback page shows TX/RX data and verification results; the I2C page shows scan progress logs and responding addresses.